Proyecto 3: Pochoco SoC de RTL a GDSII

Macro digital en GF180MCU con LibreLane

Arquitectura de Computadores
Universidad de los Andes
Semestre 2026-2

Integrantes: Nicolás Araneda Vicente Calderón Santiago Muñoz

Noviembre de 2026

Todo el código, las configuraciones y los reportes están en https://github.com/nicolasaraneda/Proyecto3_Arquitectura_De_Computadores. Los nombres de archivo, módulo y corrida son los del repositorio, y las decisiones se citan con su número en `docs/decisiones.md` (D1 a D22). Las corridas usan LibreLane 3.0.16 en Docker, el PDK gf180mcuD (revisión `54435919`) y la biblioteca `gf180mcu_fd_sc_mcu7t5v0`; el README tiene las versiones exactas, la instalación y los comandos.

La corrida final (`final`, `asic/rom/config.yaml`) usa $T = 100$ ns, `FP_CORE_UTIL` $= 40$ %, la estrategia de memoria 1 (Sección 2) y la pre-decodificación (Sección 4). Se exige cerrar timing en las nueve esquinas que analiza LibreLane: tres de parásitos (min, nom, max) por tres de proceso, voltaje y temperatura (ss a 125 °C y 4,5 V, tt a 25 °C y 5 V, ff a −40 °C y 5,5 V). Entre ellas están la típica, `nom_tt_025C_5v00`, y la más lenta, `max_ss_125C_4v50`. LibreLane solo trata como error la típica, así que el cierre en las demás se verifica con las métricas de cada esquina (`asic/verificar_cierre.py`, D14). La Tabla 1 resume el resultado.

Criterio	Resultado	Peor esquina
DRC (Magic)	0 violaciones	
DRC (KLayout, deck de GlobalFoundries)	0 marcadores en 684 reglas	
LVS (Netgen)	0 diferencias	
Antenas	0 nets	
Setup: WNS y TNS	65,22 ns y 0 ns	`max_ss_125C_4v50`
Hold: WNS y TNS	0,18 ns y 0 ns	`min_ff_n40C_5v50`

Tabla 1: Criterios de cierre de la corrida final. Fuente: `asic/rom/resultados/final/cierre.md` y `drc_klayout/resumen.md`.

1 Del RTL de FPGA al RTL de ASIC

`rtl/asic_top.v` instancia `pochoco_soc` sin nada propio de la Go Board. Sus puertos son `clk`, `rst_n` (activo en bajo), `btn[3:0]`, `led[3:0]`, `seg1[6:0]` y `seg2[6:0]` (un display cada uno, en orden {G,F,E,D,C,B,A} y activos en alto) y `spi_sclk`, `spi_mosi`, `spi_cs_n` y `spi_miso`. El reset pasa por un sincronizador de dos flip-flops que lo activa en forma asíncrona y lo libera en un flanco de reloj, así que todos los flip-flops salen del reset en el mismo ciclo (D6). Los parámetros `MemStrategy` y `DataWords` eligen la memoria de la Sección 2.

Para encontrar lo que dependía de la FPGA se corrió LibreLane sobre el RTL del Proyecto 2 (commit `1227fa5`) aplicando los reemplazos de a uno: c0a con el RTL sin cambios, c0b con el banco de registros en flip-flops y c0c con `asic_top`. La Tabla 2 lista cada construcción con el mensaje del flujo que la evidenció.

Construcción	Mensaje del flujo (archivo:línea)	Reemplazo
BRAM `SB_RAM40_4K` en `espino_register_file.v`	`%Error-MODMISSING ... SB_RAM40_4K`, el linter detiene el flujo (`c0a-verilator-lint.txt:1` y `:13`)	Banco de registros en flip-flops con lectura sincrónica, igual que la BRAM (D3, D4)
Contador de reset de encendido `por_cnt` con valor inicial	`%Warning-PROCASSINIT ... 'por_cnt'` (`c0b/verilator-lint.txt:7`) y `Set init value: \por_cnt` (`c0b/yosys-synthesis.txt:324`)	Puerto `rst_n` con sincronizador en `asic_top` (D6)
`$readmemh` dentro de un `initial` (`pochoco_ram.v:36`)	`Removed top 23 address bits ... from memory init port` (`c0b/yosys-synthesis.txt:992`): Yosys lo convierte en valor inicial de la memoria	ROM y memoria generadas por el assembler (Sección 2)
Inferencia de BRAM para la RAM de 512 palabras	`Adding EN signal on $memory\u_soc.u_ram.mem[93]` (`c0b/yosys-synthesis.txt:1469`): la memoria pasa a flip-flops; `c0b/stat.rpt`: 17 188 flip-flops y 2 327 825 µm²	Memoria dimensionada según el uso del juego (Sección 2)
SPI sin instanciar	`%Warning-UNUSEDSIGNAL ... 'spi_req'` (`c0b/verilator-lint.txt:18`)	Puertos presentes, `spi_miso` fijo en 1: el juego no usa SPI (D7)

Tabla 2: Construcciones del Proyecto 2 que dependían de la FPGA. Los archivos son copias de los logs de LibreLane en `asic/p2_original/evidencia/`.

Los dos valores iniciales son el problema de fondo. La biblioteca no tiene flip-flops con valor de encendido, así que la síntesis los descarta sin otro aviso que esas líneas. En silicio `por_cnt` partiría en un valor cualquiera, con un reset interno de entre 0 y 4095 ciclos, y la RAM partiría con instrucciones arbitrarias. Cuatro construcciones no produjeron ningún mensaje porque son valores o convenciones, no sintaxis:

- `x0` valía 0 solo porque la BRAM se inicializaba en cero y nunca se escribía. En el banco nuevo `x0` no se almacena y la lectura de la dirección 0 entrega 0 por hardware.
- Los segmentos salían invertidos (`~seg1`) porque el display de la Go Board es activo en bajo. En el macro los puertos quedan activos en alto, que es la función lógica (D8).
- `CYCLES_PER_TENTH = 2500000` en `game.s` supone el oscilador de 25 MHz de la placa. Se corrige para el macro en la Sección 4.4.
- `DebounceDivBits = 15` se eligió para 25 MHz. A 10 MHz da una muestra cada 3,3 ms y acepta un cambio en unos 10 ms, muy por debajo de la décima que muestra el juego, así que se mantuvo (D10).

2 Memoria y programa del juego

El juego usa 115 palabras de instrucciones (460 bytes) y 0 de datos. Las 115 las cuenta el assembler al ensamblar `sw/game.s`: lo informa al terminar y queda como `GAME_WORDS` en la imagen generada. Las 0 de datos salen del programa y de la simulación: todos los `lw` y `sw` usan como base `x2`, que apunta a los periféricos en `0x8000_0000`, y la simulación de la partida completa (`tb/game_tb.v`) no hace ninguna lectura ni escritura en la memoria. Las dos cantidades quedan bajo el máximo recomendado de 256 y 64. Se dejó una RAM de datos de 4 palabras en ambas estrategias (D9): conserva la RAM que nombra la estrategia 1 por 128 flip-flops, y con el mismo valor en las dos la comparación mide solo cómo se guarda el programa.

Extensión del assembler. Se agregaron dos salidas Verilog. `-rom` escribe el módulo `rtl/gen/pochoco_rom.v` completo: un `case` con una palabra por dirección, comentada con su desensamblado, sin reloj ni flip-flops, que entrega 0 fuera del programa. `-vh` escribe `rtl/gen/game_`

`image.vh` con `GAME_WORDS` y la función `game_word(i)`. `make image` regenera ambos desde `game.s`, y siete pruebas nuevas (`assembler/tests/test_image.py`, 80 en total) comprueban que coinciden con el `.hex` y con el `game.s` actual.

Estrategias. En la estrategia 1 (`rtl/pochoco_mem_rom.v`) la ROM entrega la instrucción a través de un registro que conserva la lectura registrada de la RAM del Proyecto 2, y el puerto de datos ve la RAM de 4 palabras en flip-flops. En la estrategia 2 (`rtl/pochoco_mem_ff.v`) instrucciones y datos comparten una memoria de $115 + 4 = 119$ palabras en flip-flops, y el programa entra como valor de reset: cada bit queda como un flip-flop con reset o con set según la imagen, y cualquier palabra se puede escribir desde el puerto de datos, como en el Proyecto 2. La Tabla 3 compara las dos con el mismo RTL (sin la pre-decodificación de la Sección 4) y la misma configuración del flujo.

	Estrategia 1 (ROM)	**Estrategia 2 (flip-flops)**
Área de celdas de síntesis	146 400 µm²	693 901 µm²
Área de celdas tras place and route	173 568 µm²	838 173 µm²
Flip-flops	921	4 602
Utilización real (pedida: 40 %)	47,5 %	48,5 %
Área del die	0,394 mm²	1,794 mm²
Período mínimo que cierra timing	55 ns	62 ns
Frecuencia máxima	18,2 MHz	16,1 MHz
Nets con violación de antena	0	1

Tabla 3: Estrategias de memoria a $T = 100$ ns, corridas `s1_*` y `s2_*`. El período mínimo sale de barrer `CLOCK_PERIOD` hasta dejar 2 ns entre el último que cierra y el primero que no. Fuente: `asic/barrido.md`.

La estrategia 2 tiene 3 681 flip-flops más, que son los $119 \times 32 = 3\,808$ bits de la memoria unificada menos los 128 de la RAM de datos de la estrategia 1. La ROM guarda los mismos 115×32 bits como lógica combinacional y ocupa 7 448 µm², el 4,7 % del diseño (Sección 3). La ruta crítica de la estrategia 2 es igual a la de la 1 hasta la dirección que calcula la ALU, pero sigue por la decodificación de esa dirección sobre las 119 palabras hasta un flip-flop de la memoria, en un die 4,6 veces más grande. Por área, frecuencia y antenas se eligió la estrategia 1 para la corrida final (D17). Con la configuración final del flujo (Sección 4) ocupa 182 265 µm² de celdas y 0,409 mm² de die.

3 Síntesis

La corrida final aplana el diseño salvo el register file, la ALU y el contador de ciclos, cuya jerarquía conserva con `SYNTH_KEEP_HIERARCHY_MODULES` (D12). El contador se separó en su propio módulo, `pochoco_cycle_counter`, con el mismo comportamiento. Para repartir el área entre todos los módulos se sintetizó además la misma configuración sin aplanar (`SYNTH_HIERARCHY_MODE=keep`, corrida `h2_t100`, solo hasta Yosys), cuyo reporte separa cada módulo. Las dos síntesis difieren solo en que la aplanada optimiza a través de los bordes de módulo, y por eso su área es 3,7 % menor (152 169 contra 157 980 µm²).

El módulo con más área es el register file (Tabla 5), con 73 656 µm²: 46,6 % de la síntesis jerárquica y 48,4 % de la final, donde conserva su jerarquía y mide lo mismo. El contador de ciclos ocupa 3 943 µm², el 2,5 % de la síntesis jerárquica y el 2,6 % de la final; el 61 % de su área son sus 32 flip-flops con reset y el resto el incrementador.

Antes de sintetizar se esperaba que el banco de registros dominara el área (480 bits sin celdas de memoria en la biblioteca y dos árboles de lectura de 15 a 1) y que los flip-flops fueran la celda más numerosa. Las dos cosas se cumplen (Tabla 4), y el conteo de flip-flops coincide con el RTL: 818 sin reset y 186 con reset en la síntesis jerárquica, 990 en la final. Lo que no se esperaba es el peso de los multiplexores: casi un `mux2_2` por flip-flop escrito bajo condición, porque la biblioteca no tiene flip-flops con habilitación. Tampoco se esperaba que `nand2_1` quedara a la par de `mux2_2`: la ROM, la ALU y los decodificadores, con la estrategia de área, se mapean casi enteros sobre la compuerta

Celda	Cant.	%	Qué parte del diseño la explica
`dffq_1`	804	15,7	Flip-flop sin reset. 544 son del banco de registros ($15 \times 32 = 480$ bits más los dos puertos de lectura registrados, 64), 192 de la memoria (registro de instrucción, RAM de 4 palabras y salida de datos) y 82 de la pre-decodificación. La síntesis aplanada elimina 14 que valen siempre lo mismo.
`mux2_2`	759	14,8	Multiplexor 2:1. La biblioteca no tiene flip-flops con habilitación, así que cada bit que se escribe bajo condición lleva un mux de retención delante (480 del banco, 160 de la memoria, 82 de la pre-decodificación). El resto forma los árboles de lectura del banco y los muxes de operandos.
`nand2_1`	758	14,8	Compuerta más chica de la biblioteca. Yosys, con la estrategia de área, mapea sobre ella la lógica aleatoria: ALU, alineación de la LSU, decodificadores y ROM.
`oai21_1`	309	6,0	Lógica aleatoria de la ROM, la ALU y la etapa ID.
`clkinv_1`	284	5,5	Inversor chico usado como inversor común, no solo en el reloj: banco de registros, ALU y etapa ID.

Tabla 4: Las cinco celdas más usadas de la corrida final (5 118 celdas de 34 tipos). El reparto por módulo sale de `h2_t100`. Fuente: `asic/sintesis.md`.

Módulo	Instancia	Área (µm²)	%
`espino_register_file`	`u_soc.u_core.u_rf`	73 656	46,6
`espino_id_stage` (con ALU y decodificador)	`u_soc.u_core.u_id`	29 765	18,8
`espino_alu`	`u_soc.u_core.u_id.u_alu`	10 153	6,4
`pochoco_mem_rom` (con la ROM)	`u_soc.g_mem_rom.u_mem`	27 458	17,4
`pochoco_rom`	`u_soc.g_mem_rom.u_mem.u_rom`	7 448	4,7
`pochoco_periph` (con debounce y contador)	`u_soc.u_periph`	14 409	9,1
`pochoco_cycle_counter`	`u_soc.u_periph.u_cycles`	3 943	2,5
`espino_if_stage`	`u_soc.u_core.u_if`	8 511	5,4
`espino_load_store_unit`	`u_soc.u_core.u_lsu`	2 711	1,7
Bus, sincronizador de reset y control		1 470	0,9
Total		**157 980**	**100**

Tabla 5: Área de celdas por módulo en la síntesis jerárquica `h2_t100`. Las filas con sangría están incluidas en la de arriba. Fuente: `asic/sintesis.md`.

más chica.

4 Timing

4.1 Barrido de período

La Tabla 6 barre `CLOCK_PERIOD` con el RTL y la configuración de la corrida final. El diseño cierra hasta 25 ns y no a 24 ns, donde falla por 0,004 ns en la esquina lenta. La frecuencia máxima es 40 MHz. La corrida de 28 ns se detuvo a los 40 minutos y no tiene resultado, lo que no cambia el límite porque 26 y 25 ns cierran.

Cada `CLOCK_PERIOD` puede provocar una nueva optimización del circuito: el resizer cambia el tamaño de las celdas y agrega buffers según el período, y eso cambia el área, el buffering y el placement. Aquí el área de celdas sube de 182 265 µm² a 100 ns a 189 817 µm² a 26 ns. De 26 ns para abajo el layout es idéntico (mismas celdas, área y largo de cables) y solo cambia el slack: el resizer ya no encuentra nada más que mejorar.

	Setup (ns)				Hold (ns)					
	nom_tt		max_ss		nom_tt		max_ss		Área	Cierra
T (ns)	WNS	TNS	WNS	TNS	WNS	TNS	WNS	TNS	(μm^2)	(9 esq.)
100	74,694	0	65,223	0	0,450	0	1,081	0	182 265	sí
32	17,409	0	4,395	0	0,450	0	1,081	0	183 725	sí
30	16,758	0	5,103	0	0,450	0	1,081	0	186 320	sí
26	13,160	0	1,996	0	0,450	0	1,081	0	189 817	sí
25	12,160	0	0,996	0	0,450	0	1,081	0	189 817	sí
24	11,160	0	−0,004	−0,004	0,450	0	1,081	0	189 817	no

Tabla 6: Barrido de `CLOCK_PERIOD` (corridas `final` y `f1_*`) en la esquina típica `nom_tt_025C_5v00` y la más lenta `max_ss_125C_4v50`. El área es la de celdas estándar tras place and route. Fuente: `asic/final.md`.

El límite también depende de la configuración del flujo. Con la del PDK, el resizer usaba celdas de retardo (`dly*`) como buffers, 12 de ellas en la ruta crítica a 60 ns, y se excluyeron (D15). Aun así entregaba el mismo circuito en todos los períodos (173 568 μm^2 de 100 a 40 ns en el barrido `s1_*`): su estimación de parásitos en el placement no veía violaciones, y solo el STA final, con los parásitos del ruteo, las mostraba. Se excluyó además `buf_1` y se le pidió al resizer un margen de setup de 10 ns (`PL_RESIZER_SETUP_SLACK_MARGIN`, D20). Con el mismo RTL, el período mínimo bajó de 42 a 25 ns y las violaciones de max slew de 2 201 a 546 pines. Esas violaciones no son criterio de cierre: el límite de 3 ns es la restricción del flujo, y el peor valor (5,41 ns) está dentro de los 7 ns que admite la biblioteca.

4.2 Ruta crítica

La ruta crítica a 25 ns en la esquina lenta (Tabla 7) empieza en el flip-flop `_4189_`, que guarda `alu_operator[1]` en la etapa ID, y termina en `u_soc.u_core.u_rf/_2933_`, un bit de un registro del register file. Entre medio cruza la ALU hasta `lsu_addr[1]`, la alineación del dato leído en la LSU y el mux de write-back (`rf_wdata[11]`). Es el camino de una carga: ejecución (la ALU calcula la dirección), acceso a memoria (los dos bits bajos de la dirección eligen el byte o la media palabra) y write-back.

Tramo	Celdas	Retardo (ns)	Llegada (ns)
Árbol de reloj hasta el flip-flop de inicio	3	2,66	2,66
Reloj a salida de `_4189_` (`dffq_1`, etapa ID aplanada)	1	2,34	5,00
ALU (`u_soc.u_core.u_id.u_alu`) hasta `lsu_addr[1]`	9	9,50	14,49
Alineación de la carga y mux de write-back (lógica aplanada)	12	9,74	24,23
Entrada al banco: buffer y mux de retención (`u_soc.u_core.u_rf`)	2	1,55	25,78

Tabla 7: Ruta crítica de `f1_t25` en `max_ss_125C_4v50`. El dato debe llegar antes de 26,78 ns (25 ns más la latencia del reloj de captura y la corrección por reconvergencia, menos la incertidumbre y el setup): slack de 1,00 ns. Fuente: `asic/rom/resultados/f1_t25/max_ss_125C_4v50_ruta_critica.rpt`.

El análisis estático cuenta esta ruta como de un ciclo, pero en una carga el dato de la memoria llega un ciclo después de emitirla, con el pipeline detenido y la dirección estable. Por eso registrar los dos bits bajos de la dirección al emitir la carga la saca de la ruta sin cambiar el comportamiento (cambio B2 de la Tabla 9).

4.3 Cambio al RTL: pre-decodificación

La ruta crítica de partida (`s1_t55`, antes de este cambio) iba del registro de instrucción al banco de registros pasando por el decodificador, que era su tramo más largo: 19,1 de 55,1 ns. El decode está en todas las rutas de ejecución, así que se sacó de todas a la vez (D18). En este Espino toda instrucción

pasa un ciclo esperando la lectura sincrónica del banco de registros con la instrucción ya estable. La pre-decodificación usa ese ciclo: registra las salidas del decodificador (82 bits: operación y operandos de la ALU, inmediatos, señales de salto, de la LSU y de write-back) en `rtl/espino_core/espino_id_stage.v`, y en el ciclo de ejecución el decode ya no está en la ruta. Ninguna instrucción cambia su número de ciclos.

RTL	Flujo	WNS (ns)	Área (µm²)	Flip-flops	T mínimo (ns)
Sin pre-decodificación	D15	46,76	173 568	921	55
Con pre-decodificación	D15	59,75	181 137	990	42
Sin pre-decodificación	final	54,55	174 529	921	> 26
Con pre-decodificación	final	65,22	182 265	990	25

Tabla 8: Efecto de la pre-decodificación con dos configuraciones del flujo (corridas `s1_*`, `p1_*`, `r0_*` y `f1_*`). WNS de setup en `max_ss_125C_4v50` y área de celdas, a 100 ns. Sin pre-decodificación y con el flujo final, el diseño no cierra a 26 ns (WNS −3,18 ns). Fuente: `asic/barrido.md` y `asic/final.md`.

Con la misma configuración del flujo (Tabla 8), la pre-decodificación acorta la ruta en 13,0 ns (D15) y 10,7 ns (final), y baja el período mínimo de 55 a 42 ns. Cuesta 4,4 % de área en las dos configuraciones, por los flip-flops de la pre-decodificación (69 más tras la síntesis) y sus muxes de retención. Se verificó con `make equiv`: `sw/isa_test.s`, que usa todas las instrucciones del assembler, da trazas idénticas ciclo a ciclo con el RTL anterior (`116fa20`) en escrituras al banco, accesos al bus y salidas, con las dos estrategias de memoria.

Se midieron otros tres cambios, en ramas separadas y con la configuración final (Tabla 9). El rendimiento se compara con la Ley de Hierro: tiempo = instrucciones × CPI $\times T$, o sea MIPS $= f/\text{CPI}$.

Cambio (rama)	T mín. (ns)	Área (µm²)	CPI	MIPS
Corrida final (`main`)	25	182 265	2,665	15,0
B2: offset de byte de la carga registrado en la LSU (`exp/lsu-offset`)	24	181 170	2,665	15,6
B3: comparador de saltos que lee directo del banco (`exp/branch-cmp`)	26	183 962	2,665	14,4
B4: una instrucción por ciclo (`exp/cpi`)	60	158 024	1,312	12,7

Tabla 9: Otros cambios de RTL. Área de celdas a 100 ns; CPI de la simulación de la partida completa; MIPS a la frecuencia máxima. B3 no se corrió a 25 ns y B4 no cierra a 55 ns. Fuente: `asic/exploraciones.md`.

B2 es el único que mejora el período (24 ns): saca la ALU de la ruta de la carga y la ruta crítica pasa a ser el control de la detención del pipeline. B3 no lo mueve porque la ruta del salto no era la crítica: la de la carga es más larga. B4 elimina el ciclo de espera (ROM y banco de lectura combinacional, sin registro de instrucción ni pre-decodificación) y baja el CPI a la mitad, pero toda la ruta queda en un ciclo: PC, ROM, decodificador, lectura del banco, ALU y write-back. El período sube 2,4 veces y rinde 15 % menos MIPS, aunque ahorra 13 % de área. Ninguno se incorporó a la corrida final (D22): esta ya cumple todos los criterios con margen, y cada cambio agrega lógica que verificar sin cambiar el cumplimiento de ningún criterio.

4.4 Constantes del juego para $T = 100$ ns

Con $T = 100$ ns el reloj es de $f = 1/T = 10$ MHz, y una décima de segundo dura

$$\text{CYCLES_PER_TENTH} = 0{,}1\,\text{s} \times 10\,000\,000\,\text{ciclos/s} = 1\,000\,000\ \text{ciclos}.$$

Cambian dos constantes de `game.s`: `CLK_HZ` de 25 000 000 a 10 000 000 y `CYCLES_PER_TENTH` de 2 500 000 a 1 000 000 (D10). `CLK_HZ` documenta la frecuencia y ninguna instrucción la usa. Las

esperas están en décimas (`ESPERA_TENTHS = 30` son tres segundos) y no cambian, y el tiempo se sigue mostrando en décimas. `CYCLES_PER_TENTH` se carga con `li x12` en `cyc2tenths` y en `wait_tenths`, y cada `li` se expande en `lui` más `addi`: 1 000 000 = 0x000F4240, así que `lui x12, 0xF4` y `addi x12, x12, 576` (el bit 11 de `0x240` es cero y no hace falta corregir la parte alta). Al regenerar el programa con `make image` cambian cuatro palabras de la ROM (Tabla 10).

Dirección	25 MHz	10 MHz	Instrucción a 10 MHz	Subrutina
0x0148	00262637	000f4637	lui x12, 0xf4	cyc2tenths
0x014c	5a060613	24060613	addi x12, x12, 576	cyc2tenths
0x019c	00262637	000f4637	lui x12, 0xf4	wait_tenths
0x01a0	5a060613	24060613	addi x12, x12, 576	wait_tenths

Tabla 10: Palabras de la ROM que cambian al pasar de 25 a 10 MHz. Fuente: `rtl/gen/pochoco_rom.v`, commit `5d57856`.

5 Floorplan, red de alimentación y placement

5.1 Utilización y área mínima

Se barrió `FP_CORE_UTIL` de 40 a 90 % con $T = 100$ ns (Tabla 11 y Figura 1). Entre las probadas, el área mínima con la que el diseño completa el flujo y cumple todos los criterios es la de 70 %: un die de 480,0 × 497,9 µm, 0,239 mm^2. A 80 % el placement detallado falla durante el CTS, y a 90 % las celdas ya no caben en el core. La utilización real queda 8 a 11 puntos sobre la pedida porque el core se dimensiona con el área de síntesis y después se agregan celdas de tap, buffers y el árbol de reloj.

Pedida	Die (µm)	Die (mm^2)	Real	WNS (ns)	Resultado
40 %	630,6 × 648,5	0,409	48,0 %	65,22	cierra
50 %	565,4 × 583,4	0,330	59,2 %	67,72	cierra
60 %	517,3 × 535,3	0,277	70,1 %	63,68	cierra
70 %	480,0 × 497,9	0,239	80,9 %	64,08	cierra
80 %					falla en CTS (`DPL-0036`)
90 %					falla en placement global (`GPL-0301`)

Tabla 11: Barrido de utilización a 100 ns (corridas `final` y `u1_*`), con el WNS de setup en `max_ss_125C_4v50`. Fallas: `[DPL-0036] Detailed placement failed` y `[GPL-0301] Utilization 100.531 % exceeds 100%`. Fuente: `asic/final.md` y `falla.txt` en `asic/rom/resultados/u1_u80/` y `u1_u90/`.

Las violaciones de max slew bajan de 546 pines a 40 % a 378 y 392 a 60 y 70 %, porque los cables se acortan (de 352 a 317 y 318 mm en el ruteo detallado). Las violaciones de DRC al inicio del ruteo detallado no siguen una tendencia (1 546, 2, 5 y 2 149), pero el router las lleva a cero en todas las corridas que terminan; la señal de congestión que decide el límite es que a 80 % el placement detallado ya no encuentra lugar para las celdas.

5.2 Terminales del macro

Los terminales se ubicaron por función con `IO_PIN_ORDER_CFG` (`asic/rom/pines.cfg`, D21), como se ve en el borde de la Figura 3. Al oeste van las entradas: `btn[3:0]`, `clk` al centro del lado y `rst_n`. Al este van los dos displays, cada bus contiguo y en orden de bits; al norte los LEDs y al sur el SPI. Cada grupo llegaría así a pads vecinos de un chip completo sin cruzarse, y las señales entran por un lado y salen por el opuesto. Si se deja que el flujo los ubique (corrida `u1_auto`), pone `clk` en una esquina, junta los cuatro de SPI en otra e intercala segmentos y LEDs en el lado este; ahorra solo 0,4 % de cable y el WNS es el mismo.

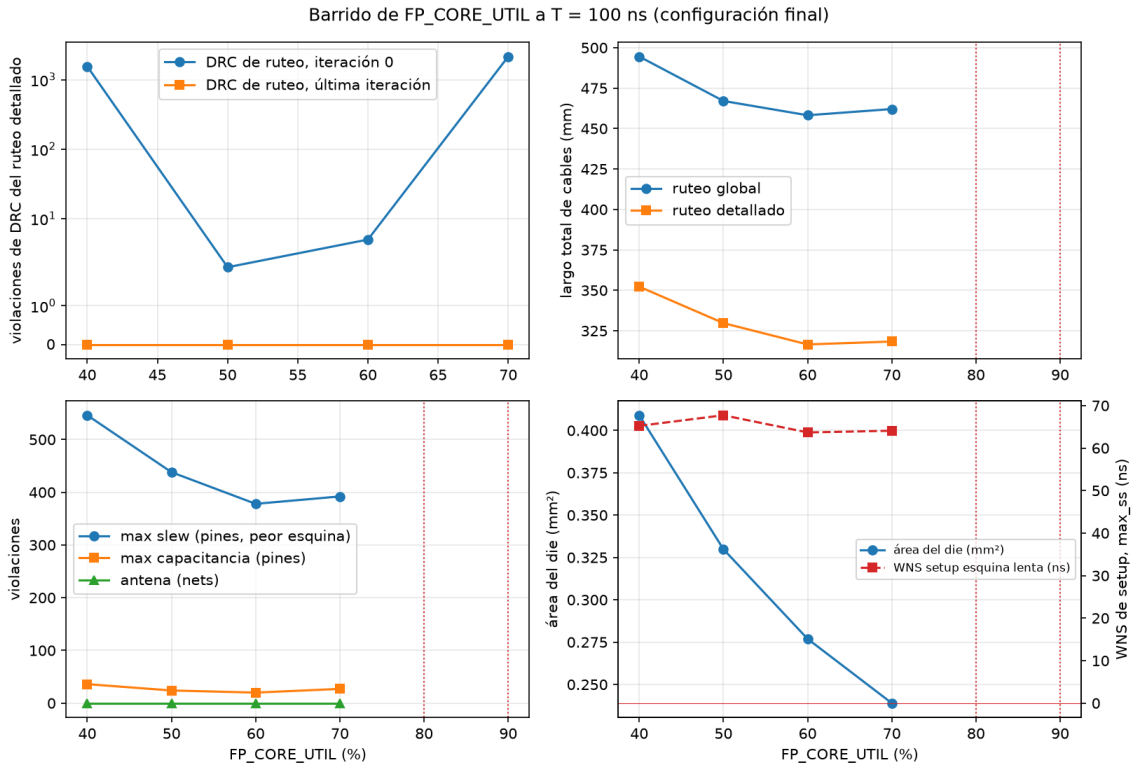


Figura 1: Utilización pedida contra violaciones de DRC del ruteo detallado, largo de cables, violaciones de slew, capacitancia y antena, área del die y WNS. Las líneas punteadas marcan las corridas que no terminan. Fuente: `asic/graficos.py`.

Estos terminales son rectángulos de Metal3 (oeste y este) y Metal2 (norte y sur) en el borde del macro, y solo conectan lógica. Un chip completo rodearía el macro con un anillo de pads de I/O: celdas grandes con protección contra descargas electrostáticas, buffers de salida capaces de entregar la corriente de un LED o un display, y entradas con histéresis.

5.3 Costo

Con US$23 000 por mm^2 y un costo proporcional al área del die, la corrida final (0,409 mm^2) costaría US$9 406, la de 70 % de utilización (0,239 mm^2) US$5 496 y la estrategia 2 (1,794 mm^2) US$41 266. La aproximación ignora que un prototipo se paga por cupo en una oblea multiproyecto con un área mínima, que el macro no incluye pads, anillo de sello ni encapsulado, que el precio por mm^2 baja con el volumen y que las máscaras son un costo fijo. Tampoco considera el rendimiento de fabricación, que baja con el área, así que el costo real por chip crece más rápido que el área.

La decisión con más impacto en área y costo fue la estrategia de memoria: con el mismo RTL y la misma configuración, la estrategia 2 necesita un die 4,6 veces mayor (Tabla 3). La siguiente es la utilización, que de 40 a 70 % reduce el die en 42 %. La pre-decodificación y la configuración del flujo suman 5 % de área.

6 Signoff y layout

6.1 DRC, LVS y antenas

La corrida final no tiene violaciones de DRC (Magic), diferencias de LVS ni nets con violación de antena (Tabla 1). Además se corrió sobre el GDS final el deck de DRC de KLayout de GlobalFoundries (684 reglas, `asic/drc/`), que LibreLane salta con este PDK, y no marcó nada. Para comprobar que

el deck detecta errores, se agregaron a una copia del GDS dos violaciones de Metal1: un trazo de 0,20 µm de ancho y dos trazos separados por 0,15 µm, contra mínimos de 0,23 µm. El deck marcó exactamente esas dos, en las reglas M1.1 y M1.2a y en sus coordenadas (`drc_klayout/control_positivo.md`).

Durante la comparación de estrategias apareció una violación de antena en la estrategia 2, en todas sus corridas. En `s2_t100`, el net `net3342` reparte una señal de alto fanout entre cinco buffers y ocupa una envolvente de 666×364 µm (Figura 2). En el pin `fanout3341/I`, la razón entre el área de Metal2 conectada y el área de la compuerta es 449,1, 1,12 veces el máximo de 400 (`asic/ff/resultados/s2_t100/antenna_summary.rpt`). Durante la fabricación, un tramo largo de metal conectado solo a compuertas acumula carga del grabado por plasma, y puede romper el óxido de la compuerta antes de que las capas superiores lo conecten a una difusión que la descargue.

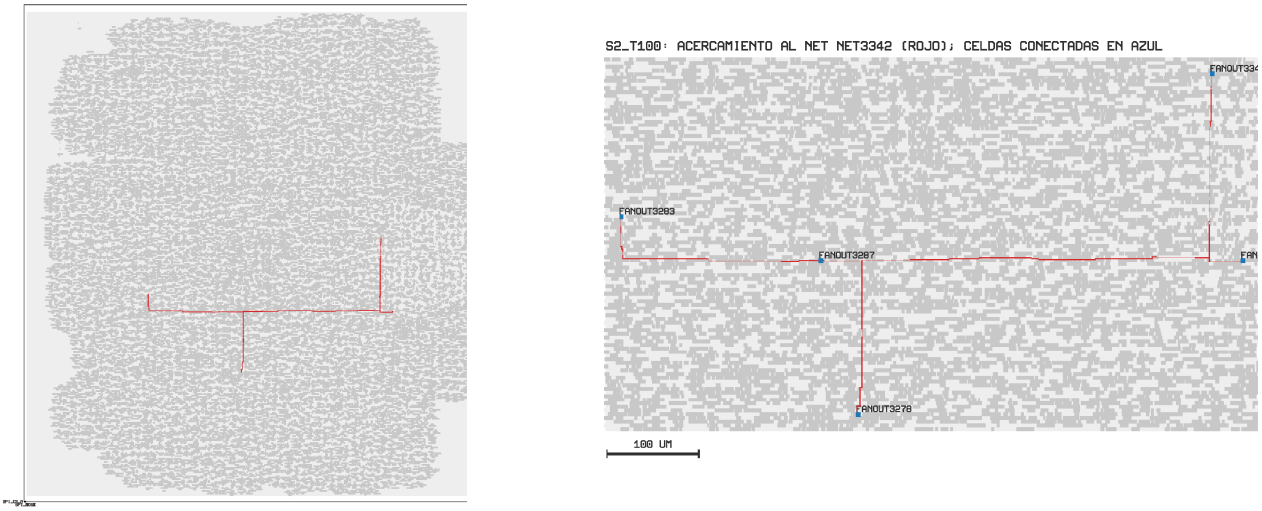


Figura 2: Net `net3342` de `s2_t100` (rojo) sobre el layout de la estrategia 2, completo y en acercamiento, con los buffers que conecta en azul. Fuente: `asic/layout/anotar.py`.

Se probaron dos soluciones (Tabla 12). La inserción heurística de diodos (`RUN_HEURISTIC_DIODE_INSERTION`) elimina la violación pero pone diodos en casi todos los nets, con 18 % más área de celdas y 16 ns menos de slack. Darle más iteraciones y margen a la reparación de antenas del ruteo global (`GRT_ANTENNA_REPAIR_ITERS = 6`, `GRT_ANTENNA_REPAIR_MARGIN = 30`) también la elimina, con 108 diodos más y sin aumentar el área. La estrategia 1 no tuvo violaciones de antena en ninguna corrida.

Corrida	Violaciones	Diodos	Área de celdas (µm²)	WNS (ns)
`s2_t100` (original)	1	150	838 173	39,65
`s2_t100_diodos`	0	36 385	992 171	23,65
`s2_t100_antrep`	0	258	833 559	30,03

Tabla 12: Violación de antena de la estrategia 2 y sus dos soluciones: nets con violación, celdas de antena, área de celdas y WNS de setup en `max_ss_125C_4v50`. Fuente: `metrics.json` y `antenna_summary.rpt` de cada corrida en `asic/ff/resultados/`.

6.2 Layout anotado

El register file, la ALU y el contador de ciclos conservan su jerarquía en la síntesis (Sección 3), así que sus celdas llegan al DEF final con el nombre de su instancia como prefijo: `u_soc.u_core.u_rf/`, `u_soc.u_core.u_id.u_alu/` y `u_soc.u_periph.u_cycles/`. Un script de KLayout (`asic/layout/anotar.py`) lee el DEF con los LEF del PDK, colorea las celdas de cada prefijo y

genera la Figura 3 y una base de marcadores (`bloques.lyrdb`) para repetir las marcas en la interfaz de KLayout (`docs/layout.md`).

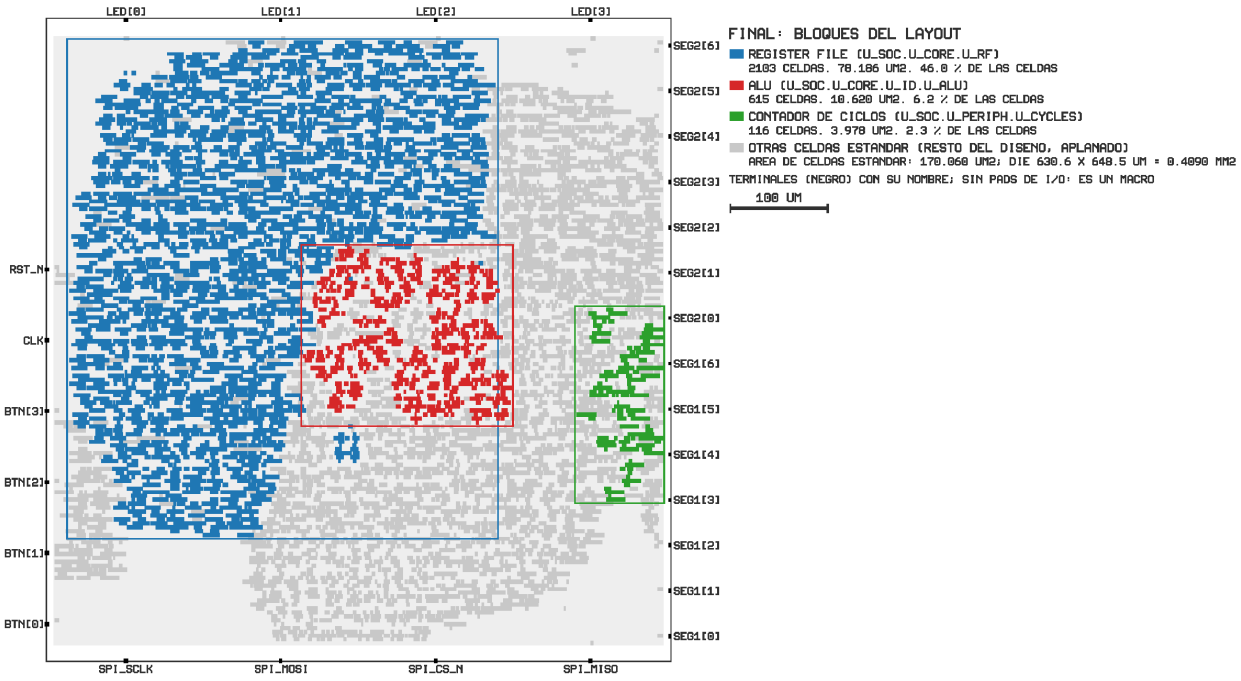


Figura 3: Layout de la corrida final con el register file (azul), la ALU (rojo) y el contador de ciclos (verde), cada uno con su rectángulo envolvente, y los terminales en el borde. Fuente: `asic/rom/resultados/final/layout/`.

El register file ocupa casi la mitad oeste del core, la ALU queda en el centro, junto a él, y el contador de ciclos en una franja angosta cerca del borde este. Las áreas de la figura son mayores que las de síntesis (78 186 contra 73 656 µm^2 en el register file) porque el resizer agrega buffers dentro de cada módulo, y sus porcentajes son sobre 170 060 µm^2 de celdas lógicas, sin las de tap ni las de borde.

7 Simulación funcional post-síntesis

`make post-synth` simula el RTL y el netlist de celdas estándar con el mismo testbench y los mismos estímulos, y compara las trazas. El netlist sale de Yosys con la configuración final (`asic/sim/config.yaml`, que solo cambia la ROM y el debounce), y las celdas se simulan con los modelos Verilog del PDK compilados sin bloques `specify` (`-DFUNCTIONAL`), o sea sin retardos. El testbench (`tb/post_synth_tb.v`) usa solo los puertos de `asic_top`, aprieta los botones en ciclos absolutos leídos de `asic/sim/estimulos.txt` y escribe una línea por cada cambio de los LEDs o del display.

Para que la simulación dure miles de ciclos y no millones, las dos usan el mismo `game.s` ensamblado con `-D CYCLES_PER_TENTH=1`, con lo que el display muestra directamente los ciclos medidos, y el debounce con `DebounceDivBits = 2`, que acepta un botón en unos 12 ciclos (`asic/sim/README.md`). El LED objetivo de cada ronda lo elige el LFSR del programa, así que los estímulos se calibraron corriendo el RTL por etapas y leyendo en su traza qué LED se encendió y cuándo (`asic/sim/calibrar.py`). Cada pulsación llega 20 ciclos después de encenderse el LED y dura 60.

Las dos trazas son idénticas, con 17 cambios de salida en 4 500 ciclos y ningún valor desconocido (X) en las salidas del netlist. La ronda correcta mide 57 ciclos en las dos simulaciones, y la pulsación incorrecta muestra EE y repite la ronda 02 en vez de pasar a la 03 (Tabla 13). Como control negativo, `make post-synth-control` atrasa 6 ciclos la pulsación de la ronda 01 solo en el netlist: el display

Ciclos	Botón	Qué pasa	Ciclo
300 a 360	`0001`	Pulsación de inicio; empieza la ronda 01	426
801 a 861	`0010`	Ronda 01, LED `0010`: acierto, el display muestra 57	1 574
2 141 a 2 201	`0010`	Ronda 02, LED `0001`: botón incorrecto, el display muestra EE	2 181
		La ronda vuelve a 02, no avanza	2 353
2 719 a 2 779	`0001`	Reintento de la ronda 02: acierto, el display muestra 57	3 492

Tabla 13: Escenario simulado. La última columna es el ciclo en que aparece cada resultado, igual en el RTL y en el netlist. Fuente: `asic/sim/resultado.md`.

muestra 70 en vez de 57 y la comparación da FAIL. Con 5 ciclos de atraso la traza no cambia, porque el programa lee los botones en un lazo de 13 ciclos y la medición no distingue pulsaciones dentro de la misma vuelta. Con las constantes reales, el netlist de la corrida final también coincide con el RTL durante 320 000 ciclos, hasta los cuatro LEDs y el 01 de la primera ronda (`make post-synth-final`). La simulación opcional con SDF de la esquina típica completó una vez con la misma traza, pero Icarus 14.0 aborta al anotar el SDF en las corridas siguientes, así que no se considera un resultado reproducible.